

Project Two LLM Transcript – Brandon Chanderban

The following outlines the interactions I had with the LLM during the completion of this assignment, arranged in the order they occurred. Questions are denoted in green.

Q: How do I remove completely the first n amount of rows within a dataframe?

To remove the first n rows of a dataframe in **R**, you can use the `slice()` function from the **dplyr** package and specify the rows you want to exclude. This is especially useful when a dataset contains metadata or extra header rows that need to be removed before analysis.

For example, if you want to remove the **first two rows** of a dataframe:

```
library(dplyr)
```

```
cleaned_data <- raw_data %>%  
  slice(-1, -2)
```

Here, the negative row indices tell R to **drop those rows from the dataframe**. The resulting object (`cleaned_data`) will contain all rows except the first two.

If the number of rows to remove is known in advance, this approach is simple and explicit. It is often used when importing datasets that contain **metadata rows above the actual column headers**, which must be removed before proceeding with data cleaning and tidying steps.

Q: Can you assist with making the following summary table more visually appealing?

```
birth_rate_summary <- birth_rate_tidy %>% group_by(year) %>% summarise(  
  average_birth_rate = mean(birth_rate, na.rm = TRUE), .groups = "drop" )  
head(birth_rate_summary)
```

Yes. `head()` is fine for checking data, but if you want it to look more polished in R, you can render it as a styled table instead.

A very clean option is `gt`:

```
library(dplyr)  
library(gt)
```

```
birth_rate_summary <- birth_rate_tidy %>%  
  group_by(year) %>%  
  summarise(  
    average_birth_rate = mean(birth_rate, na.rm = TRUE),  
    .groups = "drop"
```

)

```
birth_rate_summary %>%  
  gt() %>%  
  tab_header(  
    title = "Average Birth Rate by Year",  
    subtitle = "Summary of annual birth rates"  
  ) %>%  
  cols_label(  
    year = "Year",  
    average_birth_rate = "Average Birth Rate"  
  ) %>%  
  fmt_number(  
    columns = average_birth_rate,  
    decimals = 2  
  )
```

If you want it a little more visually appealing, you can add color shading:

```
birth_rate_summary %>%  
  gt() %>%  
  tab_header(  
    title = "Average Birth Rate by Year",  
    subtitle = "Summary of annual birth rates"  
  ) %>%  
  cols_label(  
    year = "Year",  
    average_birth_rate = "Average Birth Rate"  
  ) %>%  
  fmt_number(  
    columns = average_birth_rate,  
    decimals = 2  
  ) %>%  
  data_color(  
    columns = average_birth_rate,  
    method = "numeric",  
    palette = c("lightyellow", "orange", "red")  
  )
```

Q: The created table (in the attached image) with the following code is too cluttered:

```
gdp_summary %>% gt() %>% tab_header( title = "Average GDP by Year", subtitle =
"Summary of the annual GDP values" ) %>% cols_label( year = "Year", average_gdp
= "Average GDP" ) %>% fmt_number( columns = average_gdp, decimals = 2 )
```

How do I allow for easier visibility?

Average GDP by Year	
Summary of the annual GDP values	
Year	Average GDP
1960	71,270,852,281.24
1961	72,748,412,663.61
1962	76,395,186,608.50
1963	82,363,077,801.79
1964	90,481,675,380.50
1965	91,827,030,900.01

Yes — the main issue is that the GDP values are being displayed in their full raw form, which makes the table difficult to read. A much clearer approach is to convert the GDP values into **billions of USD** before displaying them, and then relabel the column accordingly.

A revised version would be:

```
gdp_summary %>%
  gt() %>%
  tab_header(
    title = "Average GDP by Year",
    subtitle = "Summary of annual GDP values (USD billions)"
  ) %>%
  cols_label(
    year = "Year",
    average_gdp = "Average GDP (Billion USD)"
  ) %>%
  fmt_number(
    columns = average_gdp,
    decimals = 2,
    scale_by = 1e-9
  )
```

Here, the argument `scale_by = 1e-9` divides the GDP values by one billion at the display stage, which makes the figures much more manageable and interpretable. Instead of showing very large raw numbers, the table will now present GDP values in **billions of USD**, thereby improving the clarity and overall readability of the summary table.

Q: In the following bit of R code--

```
selected_gdp_countries <- c("United States", "Guyana", "China", "India", "Brazil",  
"Nigeria")
```

```
gdp_selected <- gdp_tidy %>%  
  filter(country %in% selected_gdp_countries)
```

```
ggplot(gdp_selected, aes(x = year, y = gdp, color = country)) +  
  geom_line() +  
  labs(  
    title = "GDP Trends for Selected Countries",  
    x = "Year",  
    y = "GDP",  
    color = "Country"  
  ) +  
  theme(plot.title = element_text(hjust = 0.5))
```

– the lines corresponding to the individual countries (at least the lower lines) appear much too cluttered owing to magnitude of the US and China. Is there a way to account for this to better the visualization?

Yes. Since the GDP values for the United States and China are much larger than those of the other selected countries, a useful way to improve the visualization is to apply a logarithmic scale to the y-axis. This reduces the dominance of the highest values and makes the lower lines easier to distinguish while still keeping all countries on the same graph.

The revised code would be:

```
selected_gdp_countries <- c("United States", "Guyana", "China", "India", "Brazil",  
"Nigeria")
```

```
gdp_selected <- gdp_tidy %>%  
  filter(country %in% selected_gdp_countries)
```

```
ggplot(gdp_selected, aes(x = year, y = gdp, color = country)) +  
  geom_line() +  
  scale_y_log10() +  
  labs()
```

```

title = "GDP Trends for Selected Countries",
x = "Year",
y = "GDP (log scale)",
color = "Country"
) +
theme(plot.title = element_text(hjust = 0.5))

```

This approach preserves all six countries in a single figure while making the trends for smaller economies such as Guyana and Nigeria more visible.

Q: When I execute the following –

```

top_gdp_countries <- gdp_tidy %>% filter(year == max(year, na.rm = TRUE)) %>%
drop_na(gdp) %>%
arrange(desc(gdp)) %>%
slice_head(n = 10)

```

```

top_gdp_countries %>%
select(country, country_code, year, gdp) %>%
gt() %>%
tab_header(
  title = "Top-Performing Countries by GDP",
  subtitle = "Highest GDP values in the most recent year available"
) %>%
cols_label( country = "Country", country_code = "Country Code", year = "Year", gdp
= "GDP" ) %>%
fmt_number( columns = gdp, decimals = 2 ) -- I get the attached. Is there a way to
add a filter that only filters out countries that are actually countries (not regions). Is
there a base R function that has a list of the world countries?

```

Top-Performing Countries by GDP			
Highest GDP values in the most recent year available			
Country	Country Code	Year	GDP
World	WLD	2022	100,562,000,000,000.00
High income	HIC	2022	61,535,800,000,000.00
OECD members	OED	2022	59,596,500,000,000.00
Post-demographic dividend	PST	2022	56,013,500,000,000.00
IDA & IBRD total	IBT	2022	40,440,000,000,000.00

Yes — what you’re seeing are **aggregate groups** (e.g., *World*, *High income*, *OECD members*), not just individual countries. The cleanest fix is to filter your data against a reference list of actual country names or country codes.

The easiest approach in R is to use the **countrycode** package, since it contains standard ISO country lists.

Option 1: Filter by valid country names

```
library(countrycode)
```

```
valid_countries <- unique(countrycode::codelist$country.name.en)
```

```
top_gdp_countries <- gdp_tidy %>%  
  filter(country %in% valid_countries) %>%  
  filter(year == max(year, na.rm = TRUE)) %>%  
  drop_na(gdp) %>%  
  arrange(desc(gdp)) %>%  
  slice_head(n = 10)
```

Option 2: Filter by valid ISO-3 country codes

This is often even better, since country codes are more standardized than names.

```
library(countrycode)
```

```
valid_codes <- unique(countrycode::codelist$iso3c)
```

```
top_gdp_countries <- gdp_tidy %>%  
  filter(country_code %in% valid_codes) %>%  
  filter(year == max(year, na.rm = TRUE)) %>%  
  drop_na(gdp) %>%  
  arrange(desc(gdp)) %>%  
  slice_head(n = 10)
```

Then your `gt()` table can remain exactly the same.

Why this works

The World Bank-style datasets often include:

- regions,
- income groups,
- development groupings,
- and world totals,

all of which look like “countries” structurally, but are not actual sovereign countries.

Filtering against an ISO list removes those aggregate entries.

Q: How would I go about converting the GDP within the same summary table so that it depicts GDP in Billions of USD, for more clarity?

Yes — that would make the table much clearer.

You can create a new column that divides GDP by **1 billion** (1e9), then display that instead of the raw GDP values.

Here's the adjusted version:

```
library(countrycode)
```

```
valid_codes <- unique(countrycode::codelist$iso3c)
```

```
top_gdp_countries <- gdp_tidy %>%
  filter(country_code %in% valid_codes) %>%
  filter(year == max(year, na.rm = TRUE)) %>%
  drop_na(gdp) %>%
  arrange(desc(gdp)) %>%
  slice_head(n = 10) %>%
  mutate(gdp_billions_usd = gdp / 1e9)

top_gdp_countries %>%
  select(country, country_code, year, gdp_billions_usd) %>%
  gt() %>%
  tab_header(
    title = "Top-Performing Countries by GDP",
    subtitle = "Highest GDP values in the most recent year available"
  ) %>%
  cols_label(
    country = "Country",
    country_code = "Country Code",
    year = "Year",
    gdp_billions_usd = "GDP (Billions USD)"
  ) %>%
  fmt_number(
    columns = gdp_billions_usd,
    decimals = 2
  )
```

What changed

- `mutate(gdp_billions_usd = gdp / 1e9)` converts raw GDP into **billions of USD**
- the table now displays `gdp_billions_usd`
- the label becomes **GDP (Billions USD)**

Q: In a renewable_tidy dataframe, the following sets out the unique listings of countries. Could you filter out and present to me those listings that are in fact

geographical regions and not countries (for the purpose of creating a line plot visualization?

[1] "Aruba" [2] "Africa Eastern and Southern" [3] "Afghanistan" [4] "Africa Western and Central" [5] "Angola" [6] "Albania" [7] "Andorra" [8] "Arab World" [9] "United Arab Emirates" [10] "Argentina" [11] "Armenia" [12] "American Samoa" [13] "Antigua and Barbuda" [14] "Australia" [15] "Austria" [16] "Azerbaijan" [17] "Burundi" [18] "Belgium" [19] "Benin" [20] "Burkina Faso" [21] "Bangladesh" [22] "Bulgaria" [23] "Bahrain" [24] "Bahamas, The" [25] "Bosnia and Herzegovina" [26] "Belarus" [27] "Belize" [28] "Bermuda" [29] "Bolivia" [30] "Brazil" [31] "Barbados" [32] "Brunei Darussalam" [33] "Bhutan" [34] "Botswana" [35] "Central African Republic" [36] "Canada" [37] "Central Europe and the Baltics" [38] "Switzerland" [39] "Channel Islands" [40] "Chile" [41] "China" [42] "Cote d'Ivoire" [43] "Cameroon" [44] "Congo, Dem. Rep." [45] "Congo, Rep." [46] "Colombia" [47] "Comoros" [48] "Cabo Verde" [49] "Costa Rica" [50] "Caribbean small states" [51] "Cuba" [52] "Curacao" [53] "Cayman Islands" [54] "Cyprus" [55] "Czechia" [56] "Germany" [57] "Djibouti" [58] "Dominica" [59] "Denmark" [60] "Dominican Republic" [61] "Algeria" [62] "East Asia & Pacific (excluding high income)" [63] "Early-demographic dividend" [64] "East Asia & Pacific" [65] "Europe & Central Asia (excluding high income)" [66] "Europe & Central Asia" [67] "Ecuador" [68] "Egypt, Arab Rep." [69] "Euro area" [70] "Eritrea" [71] "Spain" [72] "Estonia" [73] "Ethiopia" [74] "European Union" [75] "Fragile and conflict affected situations" [76] "Finland" [77] "Fiji" [78] "France" [79] "Faroe Islands" [80] "Micronesia, Fed. Sts." [81] "Gabon" [82] "United Kingdom" [83] "Georgia" [84] "Ghana" [85] "Gibraltar" [86] "Guinea" [87] "Gambia, The" [88] "Guinea-Bissau" [89] "Equatorial Guinea" [90] "Greece" [91] "Grenada" [92] "Greenland" [93] "Guatemala" [94] "Guam" [95] "Guyana" [96] "High income" [97] "Hong Kong SAR, China" [98] "Honduras" [99] "Heavily indebted poor countries (HIPC)" [100] "Croatia" [101] "Haiti" [102] "Hungary" [103] "IBRD only" [104] "IDA & IBRD total" [105] "IDA total" [106] "IDA blend" [107] "Indonesia" [108] "IDA only" [109] "Isle of Man" [110] "India" [111] "Not classified" [112] "Ireland" [113] "Iran, Islamic Rep." [114] "Iraq" [115] "Iceland" [116] "Israel" [117] "Italy" [118] "Jamaica" [119] "Jordan" [120] "Japan" [121] "Kazakhstan" [122] "Kenya" [123] "Kyrgyz Republic" [124] "Cambodia" [125] "Kiribati" [126] "St. Kitts and Nevis" [127] "Korea, Rep." [128] "Kuwait" [129] "Latin America & Caribbean (excluding high income)" [130] "Lao PDR" [131] "Lebanon" [132] "Liberia" [133] "Libya" [134] "St. Lucia" [135] "Latin America & Caribbean" [136] "Least developed countries: UN classification" [137] "Low income" [138] "Liechtenstein" [139] "Sri Lanka" [140] "Lower middle income" [141] "Low & middle income" [142] "Lesotho" [143] "Late-demographic dividend" [144] "Lithuania" [145] "Luxembourg" [146] "Latvia" [147] "Macao SAR, China" [148] "St. Martin (French part)" [149] "Morocco" [150] "Monaco" [151] "Moldova" [152] "Madagascar" [153] "Maldives" [154] "Middle East, North Africa,

Afghanistan & Pakistan" [155] "Mexico" [156] "Marshall Islands" [157] "Middle income" [158] "North Macedonia" [159] "Mali" [160] "Malta" [161] "Myanmar" [162] "Middle East, North Africa, Afghanistan & Pakistan (excluding high income)" [163] "Montenegro" [164] "Mongolia" [165] "Northern Mariana Islands" [166] "Mozambique" [167] "Mauritania" [168] "Mauritius" [169] "Malawi" [170] "Malaysia" [171] "North America" [172] "Namibia" [173] "New Caledonia" [174] "Niger" [175] "Nigeria" [176] "Nicaragua" [177] "Netherlands" [178] "Norway" [179] "Nepal" [180] "Nauru" [181] "New Zealand" [182] "OECD members" [183] "Oman" [184] "Other small states" [185] "Pakistan" [186] "Panama" [187] "Peru" [188] "Philippines" [189] "Palau" [190] "Papua New Guinea" [191] "Poland" [192] "Pre-demographic dividend" [193] "Puerto Rico (US)" [194] "Korea, Dem. People's Rep." [195] "Portugal" [196] "Paraguay" [197] "West Bank and Gaza" [198] "Pacific island small states" [199] "Post-demographic dividend" [200] "French Polynesia" [201] "Qatar" [202] "Romania" [203] "Russian Federation" [204] "Rwanda" [205] "South Asia" [206] "Saudi Arabia" [207] "Sudan" [208] "Senegal" [209] "Singapore" [210] "Solomon Islands" [211] "Sierra Leone" [212] "El Salvador" [213] "San Marino" [214] "Somalia, Fed. Rep." [215] "Serbia" [216] "Sub-Saharan Africa (excluding high income)" [217] "South Sudan" [218] "Sub-Saharan Africa" [219] "Small states" [220] "Sao Tome and Principe" [221] "Suriname" [222] "Slovak Republic" [223] "Slovenia" [224] "Sweden" [225] "Eswatini" [226] "Sint Maarten (Dutch part)" [227] "Seychelles" [228] "Syrian Arab Republic" [229] "Turks and Caicos Islands" [230] "Chad" [231] "East Asia & Pacific (IDA & IBRD countries)" [232] "Europe & Central Asia (IDA & IBRD countries)" [233] "Togo" [234] "Thailand" [235] "Tajikistan" [236] "Turkmenistan" [237] "Latin America & the Caribbean (IDA & IBRD countries)" [238] "Timor-Leste" [239] "Middle East, North Africa, Afghanistan & Pakistan (IDA & IBRD)" [240] "Tonga" [241] "South Asia (IDA & IBRD)" [242] "Sub-Saharan Africa (IDA & IBRD countries)" [243] "Trinidad and Tobago" [244] "Tunisia" [245] "Turkiye" [246] "Tuvalu" [247] "Tanzania" [248] "Uganda" [249] "Ukraine" [250] "Upper middle income" [251] "Uruguay" [252] "United States" [253] "Uzbekistan" [254] "St. Vincent and the Grenadines" [255] "Venezuela, RB" [256] "British Virgin Islands" [257] "Virgin Islands (U.S.)" [258] "Viet Nam" [259] "Vanuatu" [260] "World" [261] "Samoa" [262] "Kosovo" [263] "Yemen, Rep." [264] "South Africa" [265] "Zambia" [266] "Zimbabwe"

Yes — from your list we should only keep **true geographic regions**, not:

- individual **countries**
- **income groups**
- **development classifications**
- **institutional groupings** (OECD, IBRD, etc.)

Based on the values you provided, the **clean geographic regions** you can use are:

```
selected_regions <- c(
  "Africa Eastern and Southern",
  "Africa Western and Central",
  "East Asia & Pacific",
  "Europe & Central Asia",
  "Latin America & Caribbean",
  "Middle East, North Africa, Afghanistan & Pakistan",
  "North America",
  "South Asia",
  "Sub-Saharan Africa"
)
```

Recommended subset for a clean plot

To keep the graph readable, I recommend using **5–6 regions**:

```
selected_regions <- c(
  "Africa Eastern and Southern",
  "East Asia & Pacific",
  "Europe & Central Asia",
  "Latin America & Caribbean",
  "North America",
  "South Asia"
)
```

Avoid using these (even though they appear)

These are **not geographic regions**:

Examples from your list:

- High income
- Low income
- OECD members
- World
- IDA only
- Upper middle income
- Early-demographic dividend
- Small states

Those are **economic or institutional classifications**, not geographic areas.

Q: In creating a line plot of the six examined geographical regions with the following code –

```
{r renewable-region-comparison-plot}
```

```
ggplot(renewable_regions, aes(x = year, y = renewable_energy_pct, color = region))
```

```
+
```

```
geom_line() +
```

```
labs(
```

```

title = "Renewable Energy Usage Across Selected Regions (2013–2022)",
x = "Year",
y = "Renewable Energy Consumption (%)",
color = "Region"
) +
theme(plot.title = element_text(hjust = 0.5)) – it appears to be too cluttered, with
even the title being cut off in the preview. Is there a way to improve the legibility of
the graph? Maybe the title can be stacked?

```

Yes — the legibility of the graph can be improved quite a bit through a few small adjustments. In this case, the best fixes are to **stack the title onto two lines**, **move the legend to the bottom**, and **increase the figure dimensions** so that the plot has more room when rendered. It also helps to slightly tighten the y-axis and add points to the line so that the yearly changes are easier to follow.

A revised version of the plot would be:

```

{r renewable-region-comparison-plot, fig.width=10, fig.height=6}

ggplot(renewable_regions, aes(x = year, y = renewable_energy_pct, color = region)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.3) +
  scale_y_continuous(limits = c(9, 67)) +
  labs(
    title = "Renewable Energy Usage Across Selected\nRegions\n(2013–2022)",
    x = "Year",
    y = "Renewable Energy Consumption (%)",
    color = "Region"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16),
    legend.position = "bottom",
    legend.title = element_text(size = 12),
    legend.text = element_text(size = 10)
  )

```

Here, `\n` is used within the title to stack it across multiple lines, which prevents it from being cut off in the preview. The `fig.width` and `fig.height` chunk options provide more space for the graph when rendered, while moving the legend to the bottom helps reduce horizontal crowding. Adding `geom_point()` also makes the year-to-year changes easier to distinguish, and setting `scale_y_continuous(limits = c(9, 67))` removes unnecessary white space and improves the visibility of the trend lines.